SOFTWARE REQUIREMENTS SPECIFICATION

Trie-Based Autocomplete Engine

An Optimized Dynamic Back-End Solution for Real-Time Prefix Path Resolution and Efficient Suffix Exploration

M. Aaliyan Ahmed
FA24-BSE-123 (Leader)

Syed Abdullah Jawwad
FA34-BSE-083

Alisha Laraib
FA24-BSE-121

Ayesha Safeer
SP24-BSE-032

PURPOSE & PROJECT SCOPE

System Purpose

Specify the functional and non-functional specifications of an optimized, high-throughput dynamic Trie framework.

Designed to ingest, process, and query thousands of character nodes in real-time, executing prefix validation with sub-millisecond timelines.

Algorithmic Scope

The core engine prioritizes dynamic structural efficiency with $O(L)$ lookup performance, replacing standard linear database searches.

Ideally designed for integration as a performance-critical subsystem inside IDEs, console utilities, or search interfaces.

REAL-WORLD PROBLEMS SOLVED



Slow Database Search

Traditional search mechanisms execute slow linear checks $O(N)$ across huge datasets, leading to high latency spikes as storage arrays grow.



High CPU Server Load

Continuous database queries for dynamic recommendations overload remote servers. Running calculations locally saves bandwidth and processing power.



Poor User Experience

Delays between dynamic key presses and autocomplete selections diminish workflow typing speed, increasing overall human entry error rates.

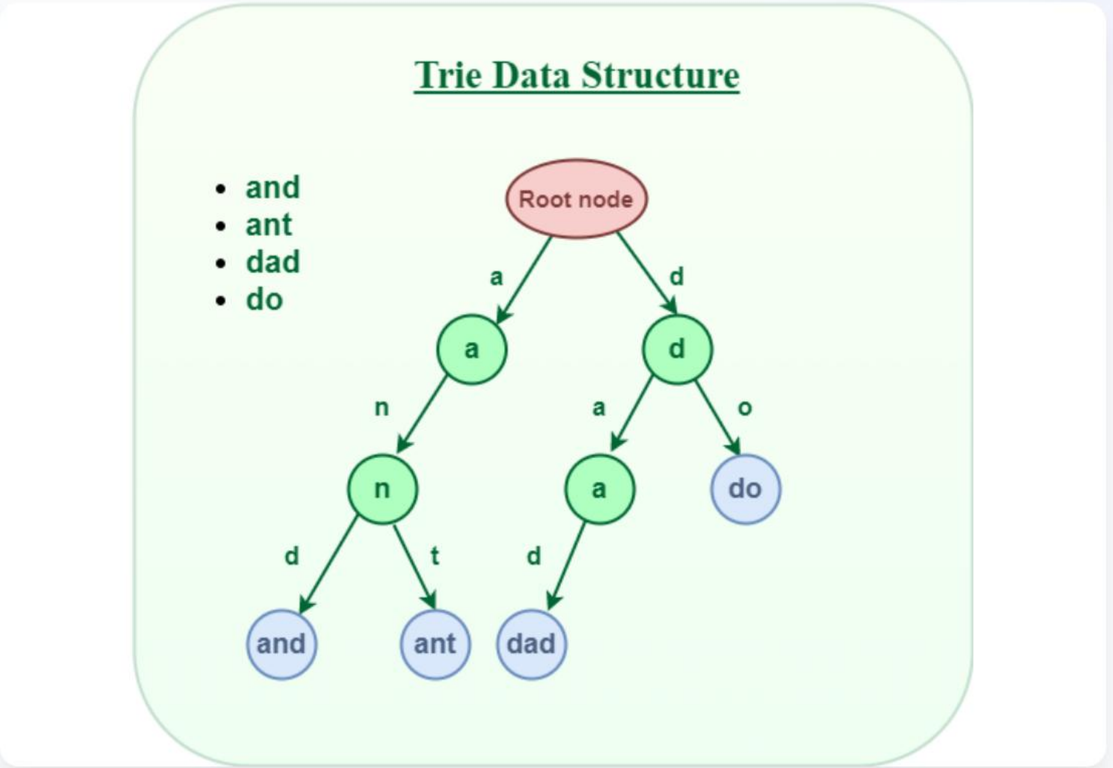
THE TRIE DATA STRUCTURE

Why Prefix Trees?

A Trie is a specialized, multi-way search tree where nodes represent key characters. Sharing common paths minimizes visual redundant spacing and simplifies search processes.

Instead of scaling with total entries (N), query traversal scales directly with prefix word length (L), ensuring deterministic execution times.

This structural behavior handles complex prefixes elegantly, mapping pathways dynamically inside system RAM for microsecond access velocities.



CORE FUNCTIONAL SPECIFICATIONS

- **Dataset Ingestion & Populating (FR-1):** Reads dictionary structures directly from external files (.txt / .csv), creating structured nodes inside system memory.
- **Case-Insensitive Normalization (FR-2):** Implicitly converts all character streams to matching lowercase standards, preventing lookup errors caused by capitalization mismatches.
- **Real-Time Prefix Resolution (FR-3):** Traverses characters one-by-one; if any navigation pointer resolves as null, the program halts immediately, optimizing resource usage.

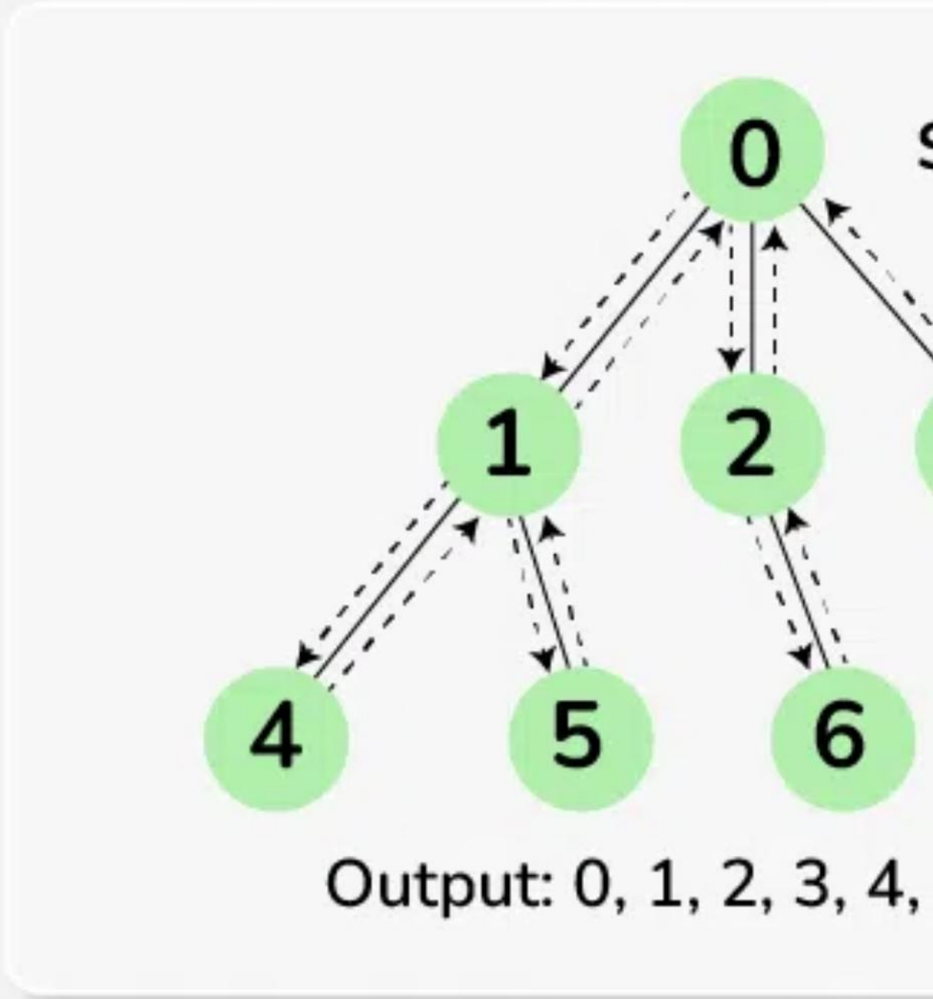
PREFIX & SUFFIX PROCESSING

Suffix DFS Exploration (FR-4)

When a prefix match is verified, a Depth-First Search (DFS) runs from that root down to the leaf node markers, compiling all valid completions.

Suggestions Capping (FR-5)

To avoid flooding command interfaces, user thresholds are enforced. Results are capped to top preferences, like a maximum of 5 or 10 words.



SYSTEM EXECUTION FLOW

1. Ingestion

Loads flat files and cleans non-alphabetical punctuation tokens.

2. Construct

Builds the in-memory Trie recursively, path-by-path.

3. Resolve

Resolves incoming key coordinates via pointer matching checkouts.

4. Search & Cap

Performs DFS, filters matching words, and caps output lists.

SYSTEM ARCHITECTURE & MODULES

User Interface (UI) Module

Interacts directly with the operator via CLI, capturing active keystrokes and updating output streams dynamically.

Engineered for modularity, allowing easy future transitions to GUI, Web, or mobile rendering layers without breaking core libraries.

Trie Data Structure Module

Coordinates underlying core operations, holding character pointers and resolving word states.

Handles structured word additions and cleanups while protecting standard memory allocation limits across pointers.

CORE INTEGRATION MODULES



Search Module

Coordinates dynamic traversals, locating starting prefix markers and running DFS workflows recursively.



File Handling

Manages initialization steps, reading database vocabulary inputs into dynamic memories at startup.



Validation & Rules

Filters invalid inputs, rejecting numeric and symbolic tokens to prevent segmentation faults.

ALGORITHMIC VERIFICATION PARAMETERS

OPERATION TYPE	TARGET TIME COMPLEXITY	RESOURCE SPACE COMPLEXITY	PERFORMANCE VARIABLE DEFINITION
Word Insertion Pipeline	$O(L)$	$O(N \times L)$	Scales linearly with maximum character word length
Prefix Route Analysis	$O(L)$	$O(1)$	Requires constant auxiliary memory footprint checks
Suffix DFS Exploration	$O(L + M)$	$O(M)$	Dependent on total matches found (M) inside matching branches

SUB-MILLISECOND PERFORMANCE TARGETS

<50ms

Maximum Dynamic Search Latency

Preserving Smooth Typing Sensations

To deliver seamless real-time visual output updates, search and render cycles are restricted to a maximum 50ms processing window, well below typical human response detection limits.

Memory optimization routines ensure explicit dynamic structures are stored locally inside systems, utilizing clean recursive memory destructors on exit to avoid RAM resource leaks.

Questions & Answers

Trie-Based Autocomplete Engine Specification Review

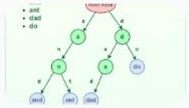
M. Aaliyan Ahmed
Group Leader

Syed Abdullah Jawwad
SRE / QA Lead

Alisha Laraib
Algorithmic Engineer

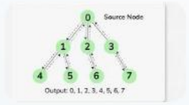
Ayesha Safeer
Data Designer

IMAGE SOURCES



https://media.geeksforgeeks.org/wp-content/uploads/20220828232752/Triedatastructure1.png

Source: www.geeksforgeeks.org



https://media.geeksforgeeks.org/wp-content/uploads/20240215172526/bfs_1.webp

Source: www.geeksforgeeks.org